

1.1.9 Bus Structure

Bus: It is a group of wires (lines or signals) which carries information from CPU to peripherals or peripherals to CPU. The CPU and Memory are connected by Data Bus, Address Bus and Control Bus.

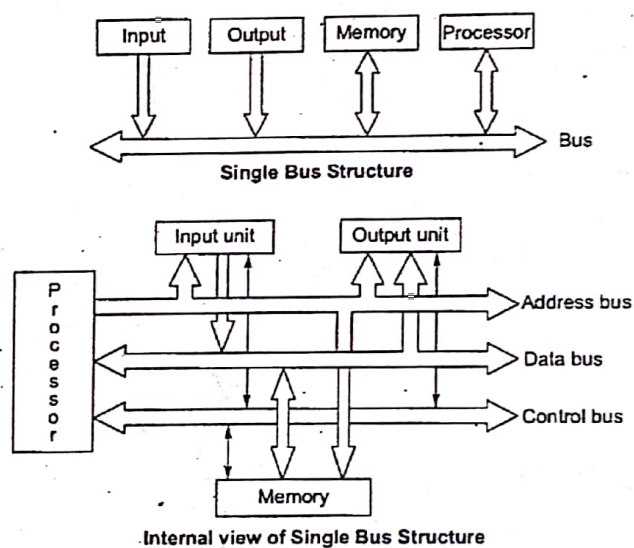
Address Bus: It is unidirectional bus which carries address information bits from processor to peripherals.

Data Bus: It is bidirectional bus which carries data information bit from processor to peripherals and vice-versa.

Control Bus: It is bidirectional bus which carries control signals from processor to peripherals and vice-versa.

1.1.10 Types of Bus Structure

- **Single bus structure:** Common bus used (shown in the following figure) to communicate between peripherals and microprocessor.

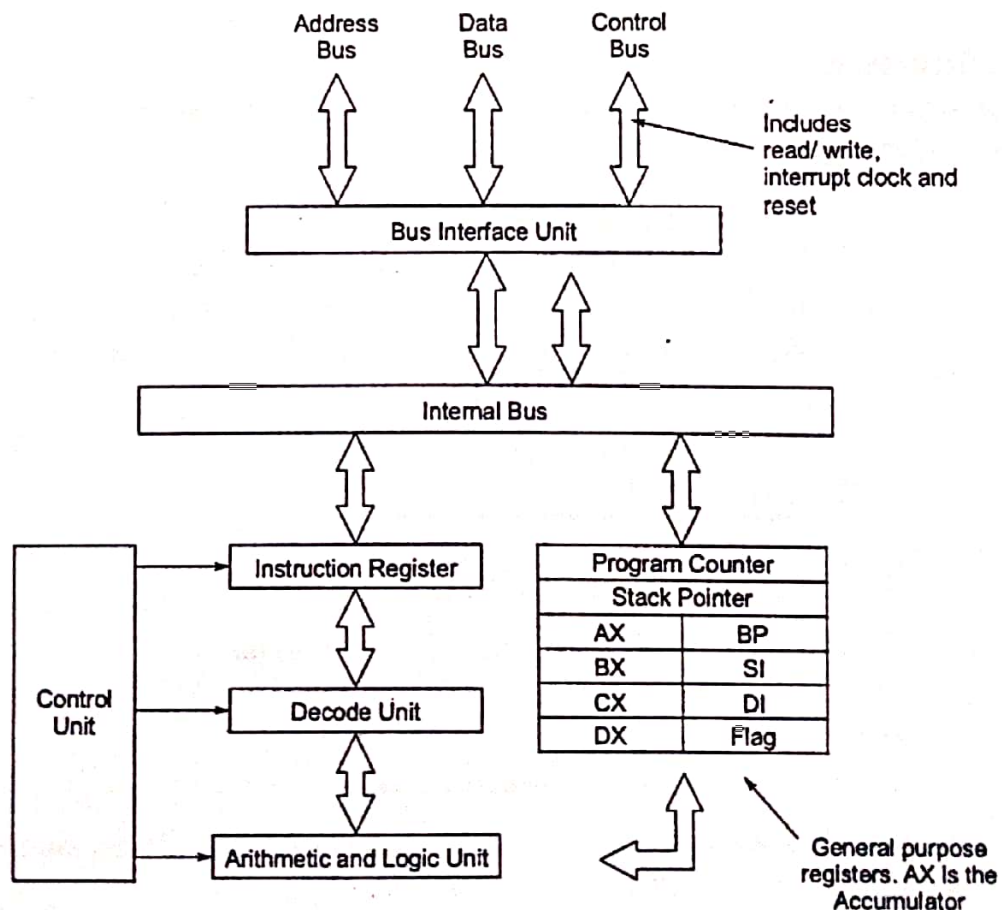


- **Two Bus Structure:** One bus can be used to fetch instruction other can be used to fetch data, required for execution. It improves the performance, but cost increases.

1.1.11 CISC and RISC Architectures

CISC (Complex Instruction Set Computers)	RISC (Reduced Instruction Set Computers)
• Large instruction set	• Compact instruction set
• Instruction formats are of different lengths	• Instruction formats are all of the same length
• Instructions perform both elementary and complex operations	• Instructions perform elementary operations
• Control unit is micro-programmed	• Control unit is simple and hardwired
• Not pipelined or less pipelined	• Pipelined
• Single register set	• Multiple register set
• Numerous memory addressing options for operands	• Compiler and IC developed simultaneously
• Emphasis on hardware	• Emphasis on software
• Includes multi-clock complex instructions	• Single-clock, reduced instruction only
• Memory-to-memory: "LOAD" and "STORE" incorporated in instructions	• Register to register: "LOAD" and "STORE" are independent instructions
• Small code sizes, high cycles per second	• Low cycles per second, large code sizes
• Transistors used for storing complex instructions	• Spends more transistors on memory registers
Examples of CISC processors: • VAX • PDP11 • Motorola 68000 family • Intel x86 architecture based processors.	Examples of RISC processors • Apple iPods (custom ARM7TDMI SoC) • Apple iPhone (Samsung ARM1176JZF) • Nintendo Game Boy Advance (ARM7) • Sony Network Walkman (Sony in-house ARM based chip)

1.1.12 General CPU Architecture (8086 Microprocessor)



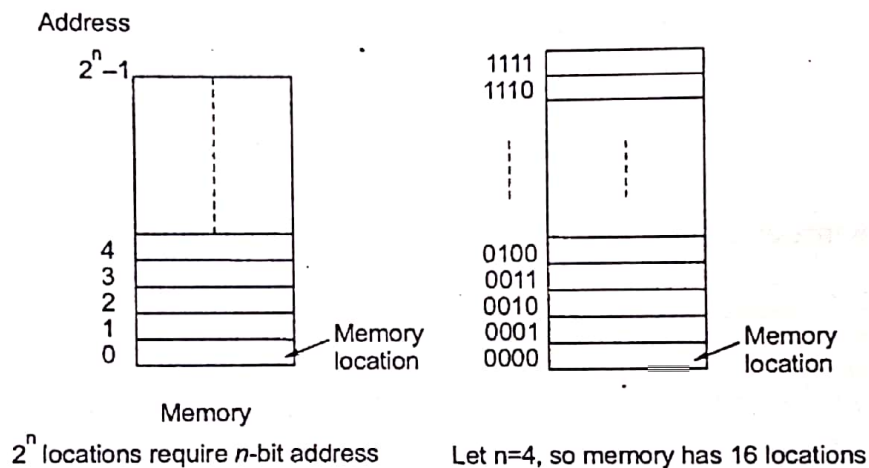
1.1.13 Issues of Computer Design

- Cannot assume infinite speed and memory.
- Speed mismatch between memory and processor
- Handle bugs and errors
- Multiple processors, processes, threads
- Shared memory
- Disk access
- Better performance with reduced power

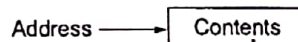
1.2 Data Storage in the Memory

How to address main memory?

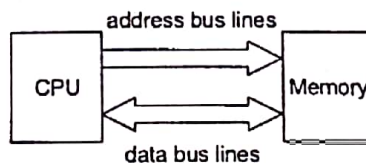
Main memory is a set of storage locations. Each location of memory has a unique address (a binary number starting from zero). Each "addressable" location holds a fixed number of bits. Any location can be accessed at high speed in any order (random access memory).



Memory consists of addressable locations. A memory location has 2 components: Address and Contents.



Data transfer between CPU and memory involves address bus and data bus

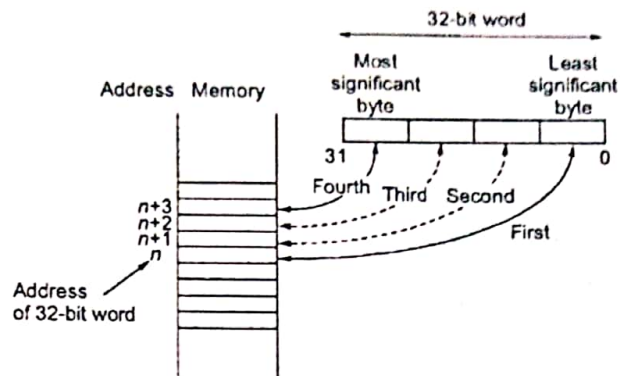


There are two ways stored information can be organized.

1. **Little Endian (little end first):** First location can hold least significant byte.

Example: Intel uses little endian

Little endian format (little end first)

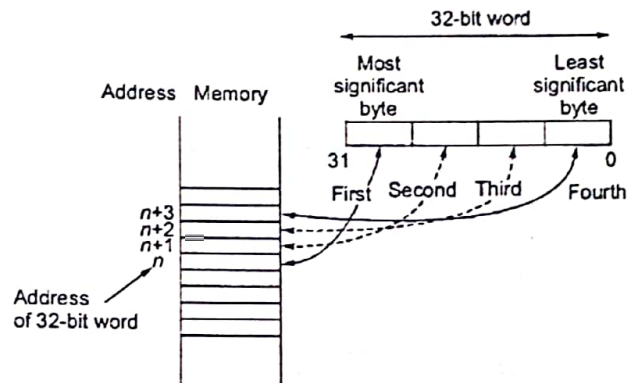


Assume 4 byte words are used in memory

2. Big Endian (big end first): First location can hold most significant byte.

Example: Mac/SUN use big endian

Big endian format (big end first)



Assume word size is 4 bytes in memory

1.3 Machine Instructions

A set of binary codes that are recognised and executed directly by a processor is called a machine code. An individual machine code is called a machine instruction.

Main memory holds machine instructions and Data. Each instruction/data require one or more memory locations depends on size of instruction/data.

The set of all codes recognized by a particular processor is known as its *instruction set*. Important things that make up memory instruction set architectures,

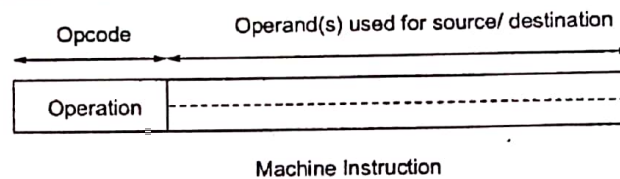
1. Memory Models (how does the memory look to the CPU)
 - (a) Addressable cell size. Most commonly is 8 bits (= 1 byte) called byte addressable
 - (b) Alignment
 - (c) Address Space
 - (d) Endianness (Little or Big endian)
2. Registers
 - (a) General Purpose Registers(to store temp results...)
 - (b) Special Purpose Registers: Program Counter (PC), Stack Pointer (SP), Input/Output Registers, Status Registers
3. Data Types
4. Instructions

Encoding Machine Instructions: Machine instruction is encoded in a binary pattern that specifies:

- (i) Operation,
- (ii) Operands/addresses used for the operation, and
- (iii) Where the result should be placed if any

Fields of an Instruction:

- (i) **Opcode:** The operation itself is usually represented by a code called the opcode.
- (ii) **Address:** Other than opcode, all the other parts of an instruction are called address or operand fields.
- (iii) **Mode:** It specifies the way the operand or effective address is determined. It may be implicit or explicit in the instruction.



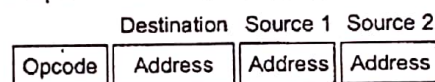
1.3.1 Instruction Formats

Based on number of address fields used in the instruction, machine instruction has the following formats:

1. Three Address Instruction
2. Two Address Instruction
3. One Address Instruction
4. Zero Address Instruction

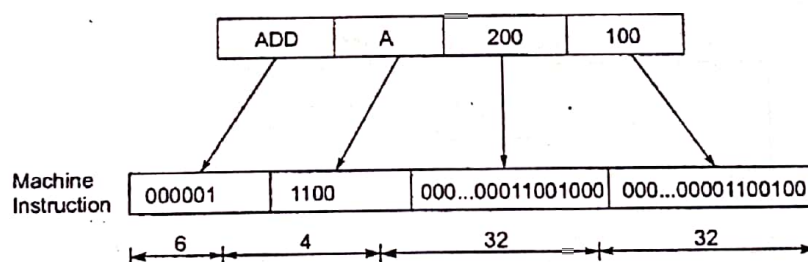
1.3.2 Three-Address Instructions

3-Address Instruction has one opcode and three address fields.



3- Address Instruction Format

Three address instruction can be encoded as following:



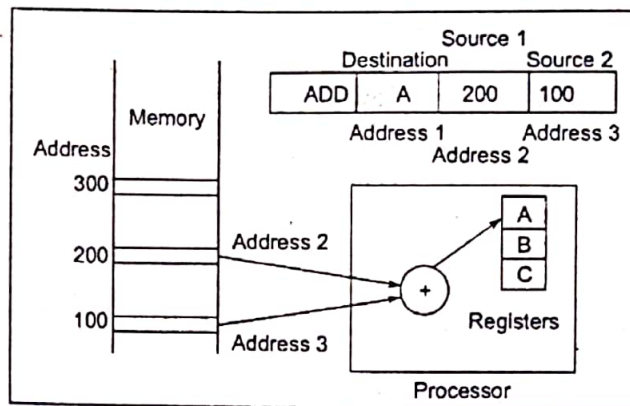
Assume 6 bits used for Opcode, 4 bits to specify a register and 32 bits for Memory address

Assume 6 bits used for Opcode, 4 bits to specify a register and 32 bits for memory address.

Example: ADD A, [200], [100]

Where [] means "contents of memory", a common notation.

ADD is the op-code *mnemonic* for Addition



Disadvantages of three address instructions:

- It has long instruction length.
- There may be three memory accesses needed for an instruction.

Example-1.1

Write the three address instructions for the following statement.

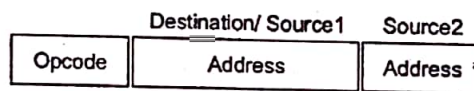
$$X = (A + B) \times (C + D)$$

Solution:

ADD R1, A, B $R1 \leftarrow M[A] + M[B]$
 ADD R2, C, D $R2 \leftarrow M[C] + M[D]$
 MUL X, R1, R2 $M[X] \leftarrow R1 \times R2$

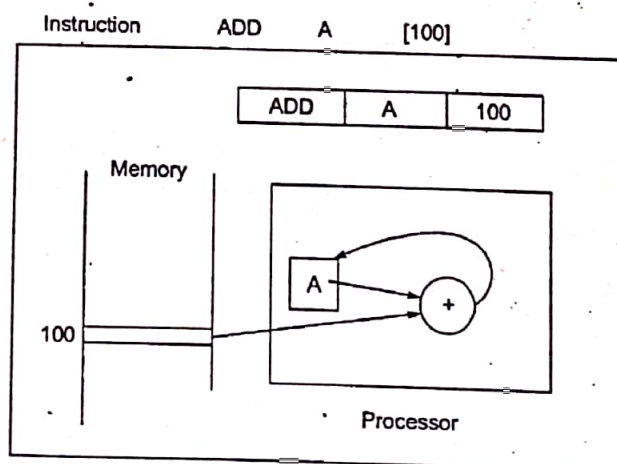
1.3.3 Two Address Instruction

2-Address Instruction has one opcode and two address fields.



Two Address Instruction Format

Example: Instruction ADD A, [100]



- **Disadvantage of two address instructions:** It may needs three memory accesses (using all operands as memory addresses for arithmetic operations, ADD [200] [100]).

Example-1.2

Write the two address instructions for the following statement:

$$X = (A + B) \times (C + D)$$

Solution:

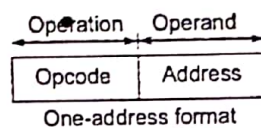
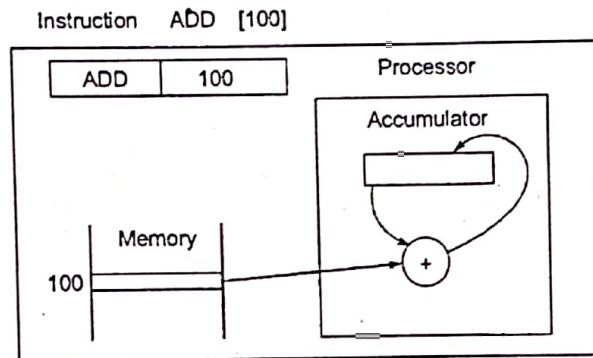
```

MOV R1, A ; R1 ← M[A]
ADD R1, B ; R1 ← R1 + M[B]
MOV R2, C ; R2 ← M[C]
ADD R2, D ; R2 ← R2 + D
MUL R1, R2 ; R1 ← R1 × R2
MOV X, R1 ; M[X] ← R1

```

1.3.4 One address Instruction

One address instruction has only two fields. One for opcode and other field for operand.

**Example:****Advantages:**

- Shorter instruction
- Eliminates two memory accesses
- Faster accessing location inside processor than memory

Disadvantages:

- Only one location for one operand and result
- Still it may need one memory access

Example-1.3

Write the one address instructions for the following statement:

$$X = (A + B) \times (C + D)$$

Solution:

```

LOAD A ; AC ← M[A]
ADD B ; AC ← AC + M[B]
STORE T ; M[T] ← AC
LOAD C ; AC ← M[C]
ADD D ; AC ← AC + M[D]
MUL T ; AC ← AC × M[T]
STORE X ; M[X] ← AC

```

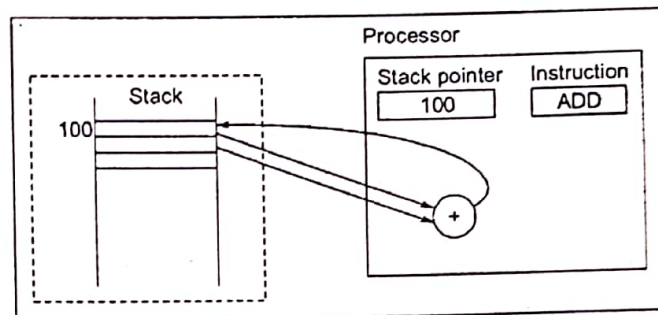
1.3.5 Zero Address Instruction

Zero address instruction has only opcode field. It has no address field.

It is possible to eliminate all addresses by using specific locations. Usually the locations for the operands/result are the top two locations of a *stack* (a last-in first-out queue) in memory or implemented with registers within the processor.

Stack pointer is a register within the processor used to hold the address of the top location. Zero-address format may be useful to compilers for producing code for arithmetic expressions (using reverse Polish notation) with the help of stack.

Example: Instruction: ADD ; it adds top two elements of the stack and stores result at top of stack



Example-1.4

Write the zero address instructions for the following statement.

$$X = (A + B) \times (C + D)$$

Solution:

```

LOAD  A ; AC ← M[A]
PUSH  A ; TOS ← A
PUSH  B ; TOS ← B
ADD   ; TOS ← (A + B)
PUSH  C ; TOS ← C
PUSH  D ; TOS ← D
ADD   ; TOS ← (C + D)
MUL   ; TOS ← (C + D) × (A + B)
POP   X ; M[X] ← TOS
    
```

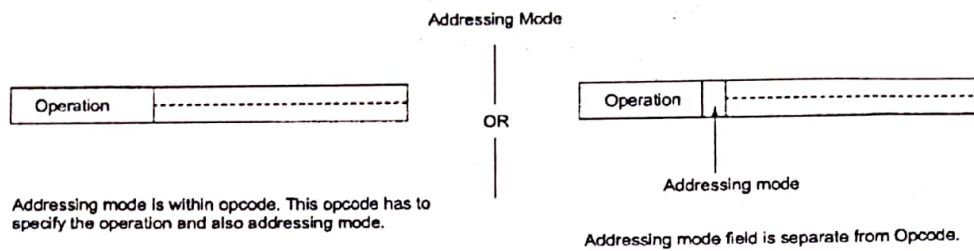
A stack-organized computer does not use an address field for the instructions ADD and MUL. The PUSH and POP instructions, however, need an address field to specify the operand that communicates with the stack.

1.3.6 Addressing Modes

The different ways in which the location of an operand is specified in an instruction are referred to as addressing modes. It is the method used to identify the location of an operand.

Addressing: "The general subject of specifying where the operands are" is called addressing.

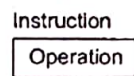
- Addressing mode may present within operation field (opcode) or in a separate **mode** field.
- The decoding step in the instruction cycle determines the operation to be performed, the addressing mode of the instruction, and the location of the operands.



Effective address: The actual address of the operand after all address computations of the addressing mode have been performed. Operand may present in register, memory, or within the instruction. Effective address of operand is computed using the addressing mode.

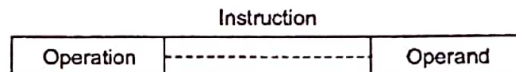
1.3.7 Types of Addressing Modes

1. **Implied Mode:** Operands are specified implicitly in the definition of the instruction.



Example: Zero-address instructions (stack operations ADD and SUB).

2. **Immediate Mode:** The actual operand (data) is specified in the instruction.

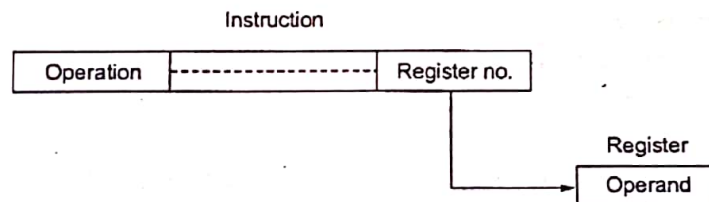


Example (Assembly): MOV R1, 123

The operand 123 is held in the instruction.

Example (High Level): $x = 123$;

3. **Register Mode:** The instruction specifies a register that has operand.

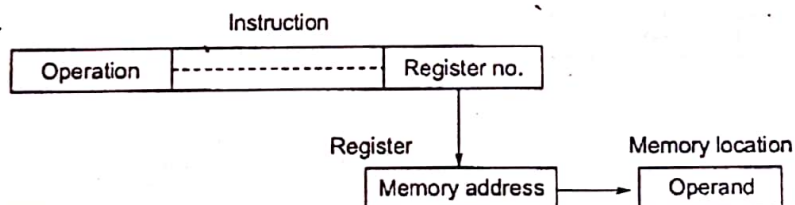


Example (Assembly): MOV R1, R2

Instruction has register R2 and R2 has operand.

Example (High Level): $x = y$;

4. **Register Indirect Mode:** The instruction specifies a register that contains the address of the operand.



The operand is held in memory. The address of the operand location is held in a register which is specified in instruction.

Example (Assembly): ADD R1, (R2)

Instruction has register R2 and R2 has memory address of operand.

Example (High Level): `int *x;`

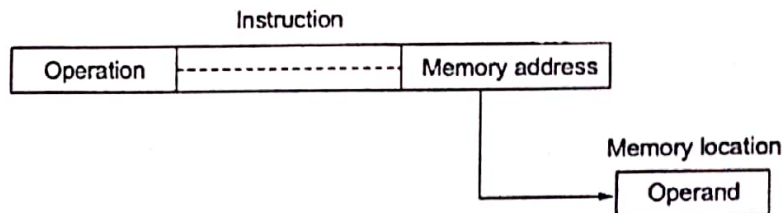
`int y, z = 0;`

`x = &y;`

`*x = 123;`

`z = z + *x; // z is similar to R1, x is similar to R2.`

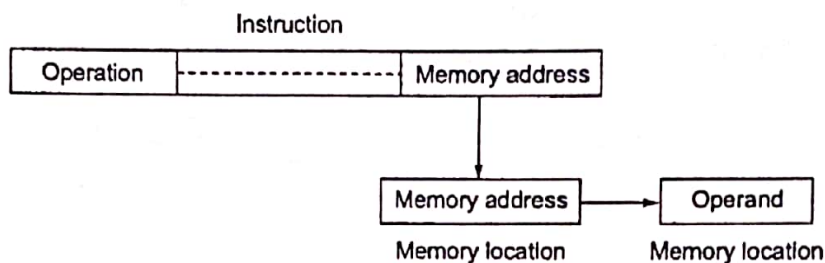
5. **Direct (Absolute) Address Mode:** The instruction has memory address where the operand is located.



Example: `MOV R1, [1000]`

The operand is in memory and the memory address 1000 of the operand is held in instruction.

6. **Indirect Address Mode:** The instruction has the address that specifies the effective address of the operand.



Example (Assembly): `ADD R1, (1000)`

Instruction has memory address 1000. At location 1000, the memory address of operand is available.

Example (High Level): `int *x;`

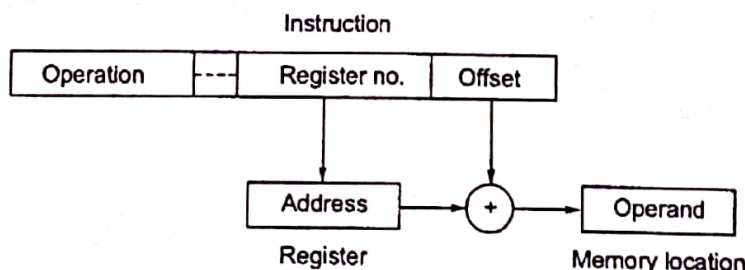
`int y, z = 0;`

`x = &y;`

`*x = 123;`

`z = z + *x; // z is similar to R1, x is similar to 1000.`

7. **Indexed Addressing Mode:** The effective address is the sum of an index register and the offset field(constant). It is similar to register indirect addressing except an offset held in the instruction is added to register contents to compute the effective address. The indexed addressing mode is useful in dealing with lists and arrays.



Example (Assembly): `MOV R1, 2(R2);` R1 stores operand from memory address: $2 + (R2)$. R2 has memory address and offset is 2. By adding constant 2 and content of R2 gives the effective address of operand.

Example (High-level): `int x[10], y; y = x[2];` // same as `y = *(x + 2);` where y is similar to R1 and x is similar to R2.

8. **Base with Index Address Mode:** The effective address is the sum of contents of base register and index register.

Example (Assembly): MOV R1, (R2, R3). Register R2 contain the base address and R3 contain the index. The effective address is the sum of the contents of registers R2 and R3.

Example (High-level): int x[10], i = 2, z; y = x[i]; // where y, x and i are similar to R1, R2 and R3 respectively.

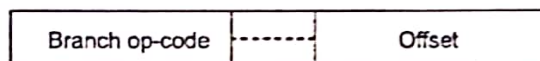
9. **Base with Index Address Mode:** It uses two registers R_i and R_j plus a constant X. The effective address is the sum of the constant X and the contents of registers R_i and R_j .

Example (Assembly): MOV R1, 2(R2, R3)

Register R2 contain the base address and R3 contain the index. The effective address is the sum of the constant value 2, the contents of registers R2 and R3.

Example (High-level): int x[10], i = 2, z; y = x[i + 3]; // where y, x and i are similar to R1, R2 and R3 respectively.

10. **Relative Address Mode:** The effective address is the sum of the address field(offset) and the content of the Program Counter (PC). Specify target location as number of instructions from branch/jump instruction. Also makes code *relocatable*. (i.e. code can be loaded anywhere in memory without altering branch/jump instructions.) The number of locations to the target is held in the instruction as an offset.



Example (Assembly): Assume PC holding 100 while executing the following instruction. JUMP 123; control will transfer to 110 by adding 100 with offset filed 23. In high-level (C) programming, using goto relative addressing can be done.

11. **Auto Increment and Auto Decrement Modes:**

(i) *Post Auto increment mode:* The effective address of the operand is the contents of a register specified in the instruction. After accessing the operand, the contents of this register are automatically incremented to point to the next item in a list. It is denoted as $(R_i)^+$.

Example: MOV R1, (R2)⁺

(ii) *Pre Auto Increment mode:* Before accessing the operand, the contents of this register are automatically incremented to point to the next item in a list. Then effective address of the operand is the incremented contents of a register. It is denoted as $+(R_i)$.

Example: MOV R1, +(R1)

(iii) *Post Auto Decrement mode:* The effective address of the operand is the contents of a register specified in the instruction. After accessing the operand, the contents of this register are automatically decremented to point to the next item in a list. It is denoted as $(R_i)^-$.

Example: MOV R1, (R2)⁻

(iv) *Pre Auto Decrement mode:* Before accessing the operand, the contents of this register are automatically decremented to point to the next item in a list. Then effective address of the operand is the incremented contents of a register. It is denoted as $-(R_i)$.

Example: MOV R1, -(R1)

1.3.8 Types of Machine Instructions

Based on operation performed, machine instructions can be divided into the following:

1. Data transfer Instructions
2. Data Manipulation Instructions (Computation): Arithmetic and Logical Instructions.
3. Program Control Instructions.

Data Transfer Instructions

Instructions that transfer data from one location (Register/Memory) to another location (Register/Memory) without changing the data. Data transfer operations supported by many processors are Load, Store, Move, Input, Output, Push, Pop, Exchange, etc.

- **LOAD:** Data transfers from memory to register.
- **STORE:** Data transfers from register to memory.
- **MOVE:** Data transfers from register to register.
- **IN:** Transfers data from input device to register.
- **OUT:** Transfers data from register to output device.
- **PUSH:** Gets data from memory or register on to the top of stack.
- **POP:** Gets data from top of stack to memory or register.
- **XCHG:** Exchanges the data between memory and registers.

Data Manipulation Instructions

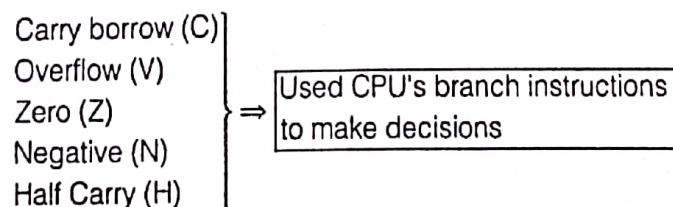
- **Arithmetic Instructions:** Performs arithmetic operations such as addition, subtraction, multiplication, division, Increment, Decrement, etc.
Example: ADD, SUB, MUL, DIV, INC, DEC, etc.
- **Logical Instructions:** Performs bit-wise logical operation such as AND, OR, exclusive-OR, NOT, shift, rotate, etc.
Example: AND, OR, NOT, XOR, SHL, SHR, ROR, ROL, etc.
- **Arithmetic and Logical Instructions:** Performs operations such as arithmetic shift left, arithmetic shift right, etc.
Example: SAL, SAR, etc.

Program Control Instructions

1. **Compare and Test Instructions:** To compare and test status flags we need support of condition code register.

Condition code register (CCR): Condition Code Register has following flag bits:

- (i) **5 status bits:** To indicate a situation after the arithmetic or logical operation.



- (ii) 2 interrupt masking bits

(iii) 1 stop disable bit

Bit Name	Function
S	Stop disable flag
X	XTRQ interrupt mask
H	Half carry from lower 4 bit
I	Interrupt mask
N	Negative
Z	Zero
V	Overflow
C	Carry

Status Bits:

- (i) **Carry Flag (C):** It indicates a carry from an addition or a borrow from a subtraction.
For addition, $C = 1$ if carry out exists, otherwise $C = 0$.
For subtraction, $C = 1$ if borrow exists, otherwise $C = 0$.
- (ii) **Overflow Flag (V):** CPU sets V flag to a logic '1' when the result of an operation is beyond of the range that can be represented.
It also called as 2's complement overflow. CPU clears flag when the number is within range.
- (iii) **Negative Flag (N):** It is also called as sign flag. Indicates whether previous arithmetic result is negative or positive. $N = 1$ for negative result, and $N = 0$ for positive result. N^{th} bit indicates state of MSB of a result. If MSB = 1 then $N = 1$, otherwise $N = 0$.
- (iv) **Half Carry (H):** Used only for BCD (Binary Coded Decimal) operations.
- (iv) **Zero Flag (Z):** Z bit is set to a logic '1' when the result is '0', otherwise $Z=0$. Note that zero is a positive number.

NOTE: N and Z flags caused by an arithmetic or a logic operation, V and C flags caused by an arithmetic operation.

Compare Instruction: Compare instruction is specifically provided, which is similar to a subtract instruction except the result is not stored anywhere, but flags are set according to the result.

Relation	$x - y$
<	Negative
>	Positive and not zero
>=	Positive or zero
<=	Negative or zero
==	Zero
!=	not zero

Example: CMP R1, R2;

Above instruction first does $(R1 - R2)$. It sets CCR flags by checking the result of $(R1 - R2)$.

2. **Unconditional branch Instruction:** It causes an unconditional change of execution sequence to a new location.

Example:

```

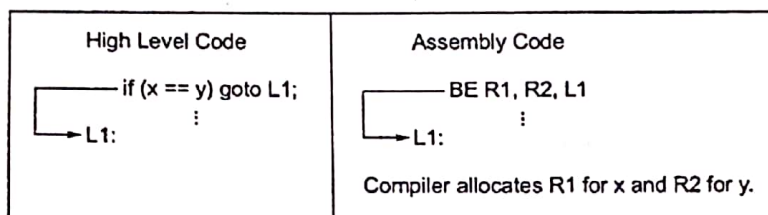
JUMP L2
MOV R3, R1
L2: ...
    
```

goto L2

3. **Conditional Branch Instruction:** A *conditional branch instruction* used to examine the values stored in the condition code register to determine whether the specific condition exists and to branch if it does.

Opcode	Condition	High level language notation
BL	Branch if less than	<
BG	Branch if greater than	>
BGE	Branch if greater or equal to	>=
BLE	Branch if less or equal to	<=
BE	Branch if equal	==
BNE	Branch if not equal	!=

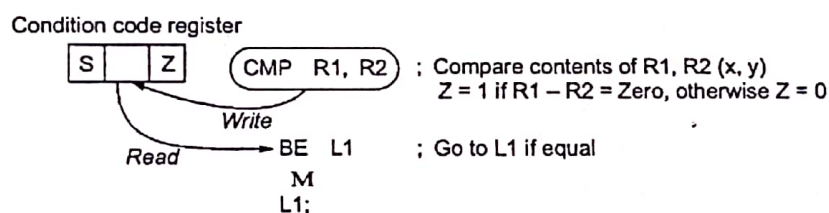
Example:



Example: Consider the following high level code.

If (x == y) goto L1;

The above code can be implemented with the following assembly code using conditional branch and compare instructions. Suppose x held in R1 and y held in R2. The if statement: Could be implemented by sequence of two instructions, CMP and BE



Example:

High Level Code	Equivalent Assembly Code
<pre> for (i = 1; i < 10; i++) a = a*i; Λ </pre>	The variable <i>i</i> is held in register R1, and the variable <i>x</i> is held in register R2. <pre> MOV R1, 1 ; i = 0 L2: CMP R1, 10 ; Compare R1 with 10 BGE L1 ; Go to L1 if end of loop MUL R2, R2, R1 ; a = a*i ADD R1, R1, 1 ; increment i JUMP L2 L1: Λ </pre>

4. **Subroutines:** A subroutine is a program fragment that lives in user space, performs a well-defined task. It is invoked (called) by another user program and returns control to the calling program when finished.

Subroutine is used for

- Frequently executed code segments
- Library routines

CALL and RET instructions are used to execute subroutines. CALL is used from the main code where subroutine invoked and RET is used to return from the subroutine to the main code.

Example: Compute the value R1 – R3 using the routine that negates the value of R3.

Subroutine	Negate the Value in R0				
	2sComp	NOT	R0, R0	R0	; flip bits
		ADD	R0, R0	R0, #1	; add one
		RET			; return to caller
Main Program	ADD	R0, R3, #0	R0		; copy R3 to R0
	JSR	2sComp	R0, #1		; negate
	ADD	R4, R1, R0			; add to R1
	Λ				

5. Halting and Interrupt Instructions:

- **NOP Instruction:** NOP is no operation, causes no change in the processor state other than an advancement of the program counter. It can be used to synchronize timing.
- **HALT:** It brings the processor to an orderly halt, remaining in an idle state until restarted by interrupt, trace, reset, or external action.
- **Interrupt Instructions:** Interrupt is a mechanism by which an I/O or an instruction can suspend the normal execution of processor and get itself serviced. Generally, a particular task is assigned to that interrupt signal. In the microprocessor based system the interrupts are used for data transfer between the peripheral devices and the microprocessor.

Example: TRAP, INTR, etc.

RESET: It reset the processor. This may include any or all of setting registers to an initial value, setting the program counter to a standard starting location (restarting the computer), clearing or setting interrupts, and sending a reset signal to external devices.

TRAP: It is non-maskable edge and level triggered interrupt. TRAP has the highest priority and vectored interrupt. In case of sudden power failure, it executes a ISR and send the data from main memory to backup memory. TRAP can be delayed using HOLD signal. This interrupt transfers the microprocessor's control to location 0024H. TRAP interrupts can only be masked by resetting the microprocessor. There is no other way to mask it.

INTR: It is level triggered and maskable interrupt. On receiving the instruction, CPU save the address of next instruction on stack and execute received instruction. It has the lowest priority. It can be disabled by resetting the microprocessor or by DI and SIM instruction.

1.3.9 Instruction Stream Vs Data Stream

There are four types of Devices bases on instruction and data streams.

1. Single Instruction Single Data
2. Multiple Instruction Single Data
3. Single Instruction Multiple Data
4. Multiple Instruction Multiple Data

Single/multiple instruction streams: How many types of instructions may be performed at once?

Single/multiple data streams: How many data streams may be operated on at once?

Most modern parallel computers (and personal computers) are MIMD.

MISD never used to large extent.

		Date Stream	
		Single	Multiple
Instruction Stream	Single	SISD Uniprocessors	SISD Pipelined Vector Processors
	Multiple	SISD Systolic arrays	SISD Multiprocessors Multi-core Multicomputers